

Preserving GPU Profiling Accuracy under Concurrent GPU-Coding Agent Workloads

Yingyi Hao¹, Xingda Wei¹, Rong Chen¹, Haibo Chen¹

¹Institute of Parallel and Distributed Systems, Shanghai Jiao Tong University

Abstract

AI agents increasingly write and tune GPU code, but each optimization step needs an accurate profile from compiling and running candidate code on a GPU. Dedicated execution preserves accuracy but leaves GPUs mostly idle, whereas uncontrolled sharing improves utilization but corrupts the feedback that guides the agent’s search.

This paper presents *atomic profiling*, which lets agents share a GPU server while making each measured interval dedicated-equivalent. PROFGUARD exploits the narrow waist of vendor timing and profiling APIs: it intercepts these APIs to find kernels and profiling operations that must run atomically, then coordinates CPU scheduling, GPU execution, and memory management so only protected intervals run with dedicated GPU access while other work continues to overlap.

We implement PROFGUARD and evaluate it with a 1.5-hour replay of an agent trace. PROFGUARD keeps p95 CUPTI performance interference below 0.3%, reduces replay time by 3.9 $\times$, raises average GPU utilization from 15.1% to 58.7%, and eliminates the five unrecovered out-of-memory failures seen under uncontrolled concurrency.

1 Introduction

The performance of GPU code—the sequences of kernels that run the computation required by AI models—determines the cost and speed of training and serving these models. Writing and tuning this code by hand, however, is expensive, for two reasons. First, each kernel must be tuned to run fast on complex and heterogeneous hardware. Second, new AI algorithms keep emerging in pursuit of greater intelligence and lower cost [7, 14], and each one requires writing new code from scratch.

Thanks to the increasing capabilities of LLM-based agents like Claude Code [1], a trend is to use these agents to automatically write efficient GPU code [11, 4, 6, 25, 13]. These agents follow the same feedback-based optimization loop as human developers: they write the code, profile it to measure performance, and loop until the performance meets expectations or the loop budget is exhausted. A profile cannot be obtained by reasoning alone: to measure a candidate, the agent must compile and actually run it on a GPU in its execution environment. Each active agent therefore occupies a GPU for as long as it keeps optimizing. In the cloud, many such agents run at the same time—for example, in post-training that issues many agents per batch, or in automated developer-agent

teams—and a system must schedule their tasks, each a single profiling run, onto a shared pool of GPUs hosted across many servers [37].

Such a system must meet three requirements. First and foremost, its profiles must be *accurate*. An incorrect profile misleads the agent into discarding fast code or wasting trials chasing a false signal; this happens even with top models such as GPT-5.5 (detailed in §3.2). Second, the system must sustain high *execution efficiency*: the agent blocks on each profile result before issuing the next trial, so this wait forms a non-trivial fraction of the agent’s end-to-end time (§3). Third, from the cluster operator’s perspective, the system must keep GPU utilization as high as possible, because GPUs remain expensive.

The typical approach prioritizes accuracy by granting each agent *dedicated* access: a server runs one task at a time, even when many tasks are scheduled onto it. No co-located work can then interfere, so every profile is accurate. Dedicated execution, however, cannot achieve high utilization and high execution efficiency together. Agent tasks are CPU-dominated, so a dedicated GPU sits mostly idle: in our dedicated replay of an agent trace, it reaches only 15.1% average utilization (§3.2). Packing more agents onto each server would improve utilization, but their tasks then queue one behind another, lengthening each agent’s wait for its profile and degrading execution efficiency; keeping efficiency high instead demands a server per agent (or a few), which—because many agents run concurrently—leaves even more GPUs idle.

The natural alternative is to run multiple agents’ tasks concurrently on each server, a standard way to raise GPU utilization [31]. It suits agent tasks especially well: with more CPUs than GPUs per server, these CPU-dominated tasks can be co-located to raise utilization while barely affecting execution efficiency, until the CPU becomes the bottleneck. Uncontrolled concurrency, however, breaks the first requirement, accuracy. Co-located tasks perturb one another’s kernels, so a profile no longer matches what a dedicated run would produce: on the same trace, concurrent execution drifts the CUPTI kernel-time profile by 2.17 $\times$ at the median and 4.94 $\times$ at p95 versus dedicated execution (§3.2).

This paper introduces *atomic profile*, which reconciles these two approaches: it lets multiple agents’ tasks share a GPU server concurrently—retaining the high utilization and execution efficiency of concurrency—while keeping every profile accurate. Profiling is *atomic* when the profiled kernels and their profiling code run as one indivisible unit,

free from any interference by co-located tasks. An atomic profile is therefore *accurate*: it is identical to the one the task would obtain on a dedicated server. We realize atomic profile in PROFGUARD, a scheduling system that coordinates the profiling intervals of co-located tasks.

Atomic GPU profiling is challenging for two reasons, and we address each accordingly. First, we must identify which region of a task’s GPU code must execute atomically, ideally in a way transparent to the task. This region is not always a single kernel: a profile interval can span multiple consecutive kernels, and any preemption of CPU or GPU work inside the interval—most likely for long-running kernels or multi-kernel intervals—corrupts the measurement. Such multi-kernel intervals are common, for example in baseline comparisons such as KernelBench [21] and in split-kernel optimizations such as split-K GEMM [19]. Unfortunately, which kernels form one atomic group depends on the application’s semantics, which the infrastructure cannot know in advance.

Our solution builds on a simple insight: although the GPU code agents write is diverse, the way it is profiled forms a narrow waist. GPU vendors officially provide only a small set of profiling utilities (e.g., CUDA events, CUPTI, and Nsight Compute), and human developers, open-source GPU projects, and the agents that learn from them all converge on these same utilities to time their kernels. Because these utilities mark where measurement starts and stops, the kernels they enclose are exactly the group that must stay atomic together. PROFGUARD therefore intercepts the utilities at this waist to read off these boundaries and identify the atomic-profile kernel group, transparently to the task, and then isolates that group while it is being profiled. This covers the CUDA-event and CUPTI-style timing paths that dominate our trace replay. A few uncommon paths, however, intentionally bypass this waist—for example, a task may use a CPU timer to fold host-side time into the measured interval, guarding against optimizations that merely move work out of the GPU-timed region. For these cases, PROFGUARD also accepts user-provided hints that explicitly mark the atomic profile boundary.

The second challenge is enforcing dedicated-equivalent execution once the profiling boundary has been identified. Naive sharing can interleave other work with the profile interval and break measurement correctness in two ways. First, CPU or GPU preemption can distort the timestamps collected by profiling code and change the GPU state observed by the profiled kernels. Second, co-located tasks can occupy GPU resources, including HBM bandwidth and VRAM capacity, so the profiled task no longer sees the same resources as in a dedicated run (§4.2).

We address these two sources of interference with two corresponding sets of techniques. Against preemption, PROFGUARD uses exclusive GPU execution and host-side priority control to make CPU/GPU scheduling around the profile interval controlled and predictable. Against resource con-

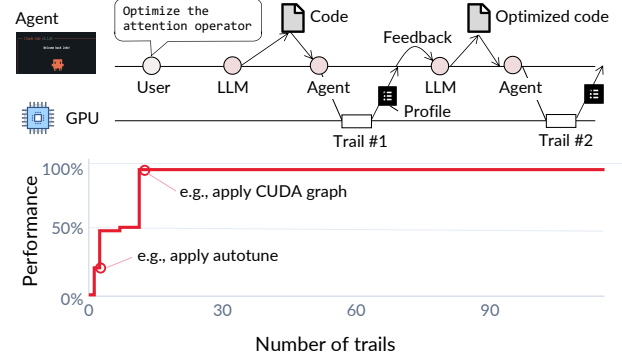


Figure 1: (Upper) An example of an agent submitting multiple trials of GPU tasks. (Lower) The goal of submitting these trials.

tention, PROFGUARD uses memory-management barriers and optimistic VRAM overcommit with replay to handle GPU-memory contention without reserving a full GPU for each task.

We implement PROFGUARD and evaluate it on a 1.5-hour replay of an agent trace. PROFGUARD meets all three requirements at once: its p95 CUPTI performance interference stays below 0.3%, while it cuts replay time by $3.9\times$ and raises average GPU utilization from 15.1% to 58.7%. It also eliminates the five unrecovered out-of-memory failures that uncontrolled concurrency leaves behind, transparently delaying allocation or replaying the few tasks that hit memory pressure during a protected interval.

2 GPU coding agent and system setup

GPU tasks in GPU coding agents. LLM-based agents are an emerging source of GPU tasks in the datacenter thanks to their ability to generate and optimize GPU code [6, 25, 13]. For example, the upper part of Figure 1 shows an agent optimizing an attention operator on a new GPU architecture. Suppose the developer has specified the optimization goal. The LLM generates a draft of the GPU operator, which is evaluated by developer-provided benchmark suite. The agent framework (e.g., Claude Code [1]) then runs one *trial* through a tool call: it sends the generated code to the GPU, executes it, and returns a performance profile. The LLM reads the profile to evaluate whether the code meets the optimization goal, e.g., whether the operator is faster than a baseline, or whether the hardware utilization is above a threshold (e.g., 80%). If not, the agent triggers the LLM to further optimize the code, and runs another trial for validation. The agent repeats this trial loop (Figure 1, lower) until the goal is met or it reaches a maximum number of trials.

In this paper, we use the term *trajectory* to denote the process where an agent finishes its development of the GPU code, including both the LLM inference time and the GPU task execution time.

Running agent-initiated GPU tasks. In large-scale agent workflows, many agents run trials at the same time, e.g.,

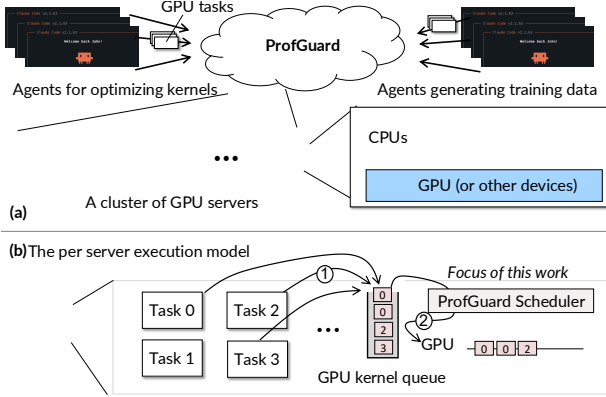


Figure 2: (a) System supporting multiple agents with short-lived GPU tasks. (b) The execution model of running these tasks on a server and target system of PROFGUARD.

when training LLM agents to write GPU code or evaluating such agents at scale [11, 4, 6, 14]. Both settings require running many agents in batches, each repeatedly generating, executing, and profiling GPU code until it meets the target or the agent reaches its time budget. To serve these concurrent tasks, AI infrastructure runs a cluster of GPU servers for agents, as shown in Figure 2 (a). Each GPU server typically has several CPUs and one GPU, but our method also apply to multi-GPU tasks (e.g., profiling multi-GPU kernels).

Targeted system and its assumptions. In this paper, we focus on the *mechanism* of running multiple tasks with accurate profiling on a single server. How to schedule these tasks—i.e., which server each task is dispatched to—is orthogonal to our work. More specifically, we assume all GPU kernels submitted by tasks on one server enter a single (logical) task queue (① in Figure 2 (b)), such that we can decide how to schedule them to execute on the GPU (②)—a common utility provided nowadays [26, 18]. Note that doing so requires no application code changes, as it works by hooking CUDA APIs [26].

3 Workload analysis and deployment tradeoffs

3.1 Workload characterizations

The studied workload. To understand GPU tasks issued by agents, we study an agent trace. Each job batches Level-2 KernelBench [21] optimization tasks: an agent repeatedly generates CUDA code, invokes a verifier to compile, execute, and profile the candidate, and then uses the returned feedback to decide the next trial. We use agent-framework logs from 98 trajectories to measure trajectory-level waiting time, and GPU-task execution logs from the same KernelBench workload to characterize per-task GPU behavior and to drive the replay experiments in §5. All tasks run under dedicated execution, which provides the reference profile for our study (§3.2). Based on this workload, we draw two workload features.

GPU task execution time matters. GPU task execution is on the agent’s critical path: after each trial, the agent must

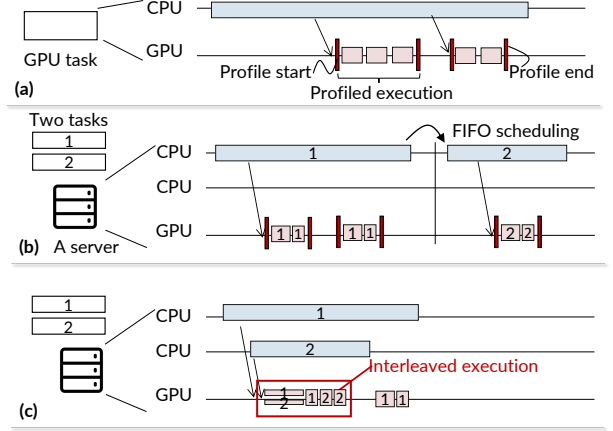


Figure 3: (a) The detailed span of a GPU task across its CPU execution, GPU execution, and the profile target. Running GPU tasks in dedicated (b) vs. concurrent (c) execution. In (c) we omit the profile targets for ease of illustration.

```

0  setup_runtime()           # torch/CUDA init
1  cases = random_test_cases() # random inputs
2  baselines = [torch_eager, torch_compile]
3  impls = baselines + [agent_kernel]
4  for case in cases:         # many input cases
5      inputs = materialize(case)
6      check_correctness(agent_kernel, inputs)
7      for impl in impls:     # profile each impl
8          fn = prepare(impl) # torch.compile / DSL JIT
9          time[case, impl] = profile(fn, inputs)
10 speedup = aggregate(base / agent)

```

Figure 4: Simplified GPU-task pseudocode from an agent trace, consisting of runtime setup, compilation, input generation, correctness checks, and profiled execution.

wait for the verifier to compile, run, and profile the generated code before it can choose the next edit. In our KernelBench workload, GPU-task execution accounts for 11.6 minutes of an average 66.3-minute trajectory, or 17.6% of the end-to-end time. Reducing this wait directly shortens the agent trajectory.

GPU time is a small fraction in a GPU task. Although these are GPU tasks, GPU time typically occupies a small fraction of the total task execution time, as shown in Figure 5 (a). Figure 4 shows the task structure behind this effect in a real GPU task: before and around the profiled execution, the task initializes the torch/CUDA runtime, generates random test inputs, checks correctness, and may compile or JIT code through `torch.compile` or DSLs such as Triton, TileLang, and CuteDSL. Once the profiling loop starts, however, each GPU-code run is short: Figure 5 (b,c) shows that one profile run in KernelBench and cudaLLM usually lasts only milliseconds or less.

3.2 System goals and current deployment methods

System goals. We set three goals for the system that supports agent-submitted GPU tasks. First, the profile must be *accurate*: a task’s profile result must match what the

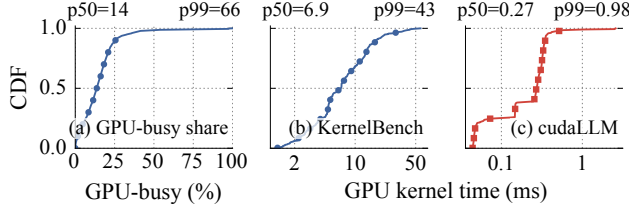


Figure 5: GPU-task workload characteristics. (a) The distribution of time spent on GPU in each task from our KernelBench execution log. (b) and (c) The detailed GPU kernel time profiled from representative agentic GPU generation benchmarks.

agent would measure if it owned a GPU server exclusively, since deployed GPU code commonly runs on an exclusively owned GPU. Modern LLM serving and training systems commonly map model computation onto a dedicated set of GPUs [10, 36, 16, 27]. Second, the task should execute as fast as possible. As shown in the previous section, GPU tasks occupy 17.6% of an average agent trajectory, so finishing them faster lets the agent complete its trajectory sooner. Finally, from the GPU cluster manager’s perspective, we should keep the GPU utilization high, i.e., use as little GPU as possible to support the tasks.

Approach #1: dedicated execution. The simplest deployment method runs one task at a time on each server. This preserves accuracy because each profile is collected in the same exclusive-GPU setting that the generated code targets in deployment. However, as shown in Figure 3 (b), the next task must wait until the current one finishes; given the CPU-dominated task structure in Figure 5, this leaves the GPU idle for much of the task lifetime.

In our end-to-end replay, this effect is large: dedicated execution uses only 79.0% CPU on average—less than one fully utilized CPU core, or 9.9% of the 8 CPU cores allocated per GPU—while average GPU utilization is only 15.1%. We describe the replay setup in §5.1.

Approach #2: concurrent execution. A natural way to improve utilization is to run several tasks concurrently on the same server (Figure 3 (c)). This overlaps the CPU-heavy parts of one task with the GPU work of another, but it also lets co-located tasks interfere with the profiled execution. We defer the interference channels to §4; here we show their end-to-end effect.

In the same replay, uncontrolled concurrency substantially shifts the profile: the median kernel-time ratio relative to dedicated execution is $2.17\times$, and the p95 ratio is $4.94\times$ (Figure 6 (a)). The extra overlap improves utilization, but it no longer preserves the feedback signal that the agent expects.

Impact on agent decisions. Profile drift can also change the agent’s search path. We test this with a controlled run of the same GPT-5.5 CUDA optimization agent and the same 100-turn budget on an FP8 GEMM task, comparing clean feedback with feedback perturbed by transient shared-GPU slowdowns. As shown in Figure 6 (b), clean feedback reaches $8.23\times$

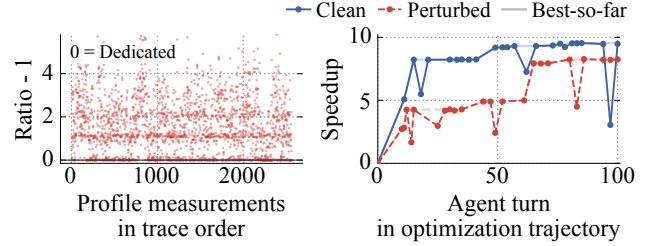


Figure 6: Uncontrolled concurrent execution shifts GPU-task profiles and can change the agent’s optimization trajectory. (a) Profile shifts measured with CUPTI on the KernelBench workload. (b) Impact of profiling interference on the agent’s optimization trajectory.

speedup by turn 15 and eventually finds $9.54\times$ speedup. With perturbed feedback, the agent reaches comparable speed only at turn 80 and ends at $8.27\times$. Thus, inaccurate profiles can corrupt the reward signal that decides which implementation the agent keeps improving.

4 Atomic concurrent profiling

PROFGUARD must make each profile *dedicated-equivalent* to meet the accuracy goal, but dedicating the whole task would sacrifice utilization and increase GPU-task latency for a fixed GPU budget. It therefore enforces *atomic profiling*: each profiled interval executes indivisibly with exclusive GPU access.

To provide atomic profiling, PROFGUARD must solve two problems. First, it must detect the fine-grained profiling interval, whose boundary is semantic: a GPU task may profile either a single kernel or a sequence of kernels, and the measurement may come from an explicit benchmark/test suite or from a profiling phase hidden inside a compiler/autotuner. Second, once the boundary is known, PROFGUARD must preserve the scheduling behavior and memory resources that the GPU task would have under dedicated execution. PROFGUARD implements these functions with the architecture in Figure 7 (e): a lightweight client runtime injected into each GPU task and a control daemon that coordinates CPU priority, GPU command queues, and VRAM scheduling.

4.1 Fine-grained atomic profiling interval detection

Problem: coarse atomicity wastes sharing opportunities. Running an entire GPU task atomically would preserve every profile, but it would also recreate dedicated execution: the GPU would be unavailable while the agent generates inputs, checks correctness, or runs CPU-heavy compiler paths. This is especially wasteful for workflows such as `torch.compile`, where only the internal benchmarking subphase needs dedicated-equivalent GPU profiling, while tracing, code generation, compilation, and autotuning setup can overlap with other tasks [22, 23].

Observation: profiling forms a narrow waist. To avoid coarse task-level atomicity, PROFGUARD must re-

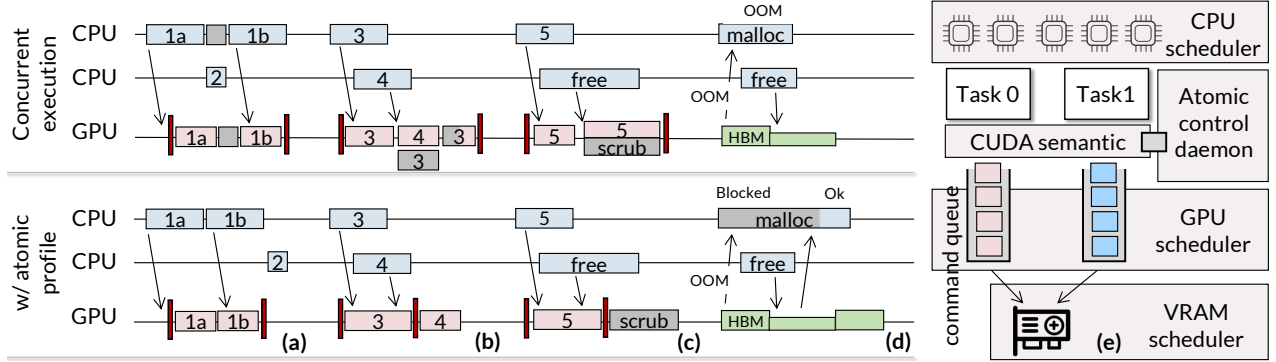


Figure 7: Four interference channels that can break atomic profiling: (a) host-side preemption, (b) GPU-side preemption and cache state drift, (c) hidden memory-management traffic, and (d) memory-capacity contention. The top half shows concurrent execution; the bottom half shows how PROFGUARD enforces atomic profiling. Panel (e) shows PROFGUARD's system architecture.

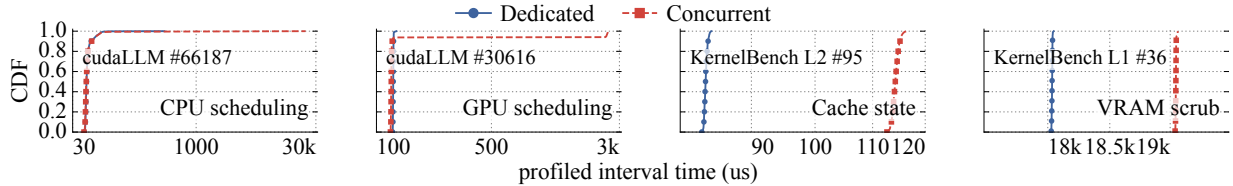


Figure 8: Controlled microbenchmarks for four interference channels. Columns isolate CPU-side scheduling delay, GPU-side scheduling and cache-state drift, background VRAM scrub traffic, and memory-capacity failures, with one representative workload per channel.

Table 1: Profiling-boundary detection in PROFGUARD.

Lock scope	Boundary hint	Profiling mechanism
Profiling range	Profiling API	CUDA events, CUPTI
CUDA context	Profiler environment	Nsight Compute / Systems
User-defined	Explicit lock hint	CPU timing, ohters

cover profiling boundaries from the task's execution stream. The key observation is that although the generated GPU code is diverse, accurate GPU benchmarking usually relies on a small set of vendor profiling interfaces, and the benchmark suites used by agents inherit these same timing harnesses. PROFGUARD therefore treats profiling API calls as interval signals. Calls such as `cudaEventRecord` and `cuptiActivityEnable` mark the start of a measured interval, while synchronization calls such as `cudaDeviceSynchronize` mark its end: by then, the benchmark has submitted the measured GPU work and is waiting for it to finish. These signals give PROFGUARD a precise region to protect: it can leave the surrounding task interleavable, while enforcing atomicity only for the profiling interval.

Coverage and integration. Table 1 summarizes the three detection paths. For CUDA events and CUPTI, the injected client runtime recognizes the profiling API semantics and locks only the measured interval. Process-level profilers such as `ncu` and `nsys` expose their presence through the process environment, so PROFGUARD conservatively protects the CUDA context. Other timing paths, including CPU timers, use an explicit lock hint. Thus, applications using a recognized profiling path require no changes to agent or profiling

code; only an unrecognized path requires a hint.

The design depends on start, stop, and synchronization semantics rather than CUDA function names. Other XPU's expose analogous event and profiling interfaces, so different XPU backends can map those calls to the same detector state machine.

4.2 Enforcing atomic profiling

After detecting a profiling interval, PROFGUARD must make that interval execute as if it were dedicated. Four interference channels can break this equivalence: CPU-side scheduling delay, GPU-side preemption and cache-state drift, background memory-management traffic, and memory-capacity contention. Figure 8 isolates these channels with controlled microbenchmarks from KernelBench [21] and cudaLLM-data [3]; each channel can shift the observed profile even when the foreground task is unchanged. PROFGUARD solves this enforcement problem by closing the four channels one by one, as summarized in Figure 7 (a–d). The rest of this section describes the four controls that PROFGUARD applies while an interval is active.

Preventing CPU preemption. The host thread remains on the critical path for kernel launches, event records, and synchronization; preemption can therefore inject launch gaps into both CUDA-event and CPU-timed profiles. During a protected interval, PROFGUARD temporarily gives the profiling task priority over co-located tasks on the host so its launch and synchronization path runs without scheduler-induced gaps. After the interval, PROFGUARD restores normal CPU scheduling and lets the tasks continue sharing the server.

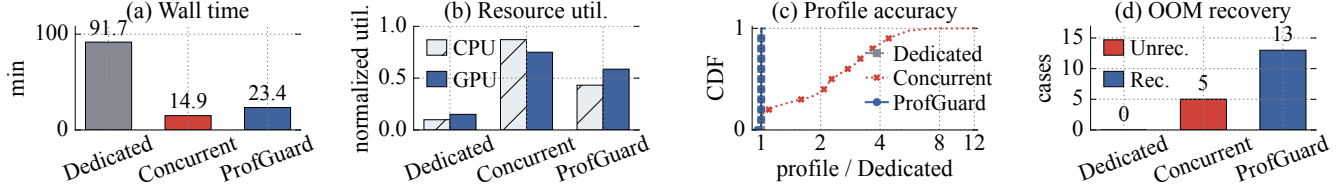


Figure 9: Replay performance and correctness on 1.5 hours of KernelBench GPU tasks: (a) wall time, (b) CPU and GPU utilization normalized by their maximum capacity on the replay server, (c) profile accuracy relative to the Dedicated baseline, and (d) unrecovered and recovered OOM cases.

Preventing GPU preemption. GPU-side preemption adds queueing and context-switch overhead, and can disturb cache residency between kernels in the same profiled sequence. PROFGUARD therefore makes the protected interval the only runnable GPU work. At interval entry, it pauses other tasks’ GPU command queues; at interval exit, it resumes them, so non-profiled work can overlap before and after the atomic interval but not inside it [2, 26].

Preventing memory-bandwidth contention. Pausing other GPU kernels is not sufficient. When a task releases VRAM, the GPU scrubs the released pages in the background. This scrub can continue after the task has been paused, consume HBM bandwidth, and interfere with the kernel being measured. Before profiling starts, PROFGUARD therefore defers new memory releases from other tasks and waits for scrub work that has already started to finish. It starts the profiling interval only after this background traffic has drained. When profiling ends, PROFGUARD resumes the deferred releases. This procedure preserves the driver’s scrub-on-free policy without allowing scrub traffic to overlap the measurement.

Preventing memory-capacity contention. The memory-release gate does not control task admission. For VRAM capacity, PROFGUARD admits tasks optimistically instead of reserving each task’s peak memory. Allocations proceed while physical VRAM is available; a task that encounters an OOM outside a profiling interval waits and retries. If the OOM occurs inside a protected interval, PROFGUARD aborts and replays that idempotent task because the interval has already lost dedicated equivalence. New tasks therefore do not face a conservative admission test: waiting or replay occurs only when actual memory pressure arises.

5 Evaluation

5.1 Experimental Setup

We evaluate PROFGUARD by replaying KernelBench GPU tasks whose dedicated execution time sums to 1.5 hours (§3.1). Each replay uses one NVIDIA H20 GPU (96 GB HBM) and eight Intel Xeon Platinum 8457C CPU cores to match the server’s aggregate CPU-to-GPU ratio. The measured intervals span 0.90–543.25 ms of aggregate kernel time; their peak memory footprints range from 32 MB to 9.94 GB. Thus, the replay covers profiling intervals with widely varying dura-

tions and memory demands. We compare three execution modes. *Dedicated* runs one GPU task at a time and provides the reference profile. *Concurrent* runs tasks concurrently without protecting profiling intervals. PROFGUARD also runs tasks concurrently, but detects and protects profiling intervals. Unless stated otherwise, Concurrent and PROFGUARD use eight workers; our scaling experiment varies the number of PROFGUARD workers. We report replay wall time, CPU and GPU utilization, profile accuracy relative to Dedicated execution, and unrecovered OOM failures.

5.2 End-to-End Throughput and Utilization

Figure 9 (a,b) shows the throughput benefit of sharing. Dedicated execution takes 91.7 minutes to replay the workload and reaches only 15.1% average GPU utilization. Uncontrolled Concurrent replay finishes in 14.9 minutes, a $6.1\times$ speedup over Dedicated, and raises average GPU utilization to 75.0%, but it does so without preserving profiling correctness (§5.3). PROFGUARD finishes in 23.4 minutes, a $3.9\times$ speedup over Dedicated, while keeping average GPU utilization at 58.7%. Thus, PROFGUARD gives up some raw overlap to make profiling intervals dedicated-equivalent, but still recovers most of the throughput lost by Dedicated execution. PROFGUARD permits overlap outside protected intervals, so the GPU remains substantially busier than under Dedicated execution while the profile-critical windows execute atomically.

Scaling with workers. Figure 10 shows that the throughput benefit is not specific to the eight-worker setting. From J1 to J8, throughput increases from 5.58 to 21.85 tasks/min ($3.91\times$), while GPU utilization rises from 15.1% to 58.7% and wall time falls from 91.7 to 23.4 minutes. The gain tapers after J4: the state breakdown shows that additional workers spend more time blocked while another worker executes an atomic profiling interval.

5.3 Profile Accuracy and Recovery

Figure 9 (c,d) shows that raw concurrency substantially distorts the profile returned to the agent. Under uncontrolled Concurrent replay, the CUPTI kernel-time ratio has median and p95 values of $2.17\times$ and $4.94\times$, while the CUDA-event benchmark-latency ratio has median and p95 values of $2.43\times$ and $5.23\times$. In contrast, PROFGUARD closely tracks Dedicated execution: p95 CUPTI performance interference stays below 0.3%, and p95 CUDA-event benchmark-latency inter-

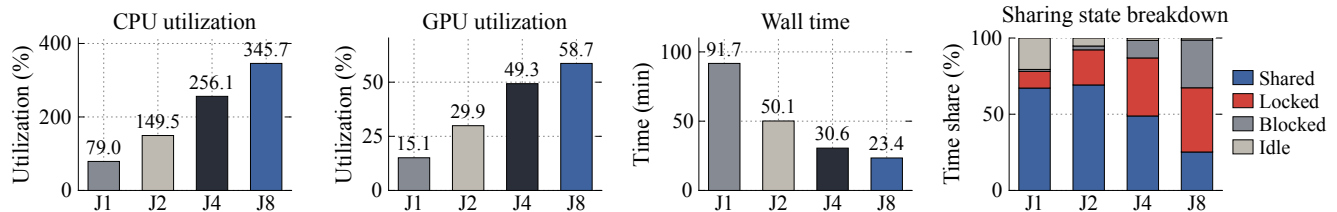


Figure 10: Worker scaling on the 512-task replay: J1 is Dedicated; J2/J4/J8 run PROFGUARD with 2/4/8 workers.

ference stays below 0.2%. Concurrent replay also exposes memory-capacity failures, with five unrecovered OOM failures. PROFGUARD encounters no unrecoverable OOM errors; the 13 GPU tasks that enter an OOM state all recover by blocking until memory becomes available and retrying allocation. These results show that PROFGUARD preserves dedicated-equivalent profile feedback while retaining the end-to-end utilization benefit of sharing.

6 Related Work

GPU sharing and scheduling. GPU sharing systems use partitioning, co-location, time sharing, preemption, migration, and placement to improve utilization. Vendor mechanisms provide concurrency or partitioning [17, 20]; cluster and runtime systems optimize throughput, latency, fairness, or tenant guarantees [31, 33, 32, 30, 15, 8, 12, 24, 34]; and preemptive schedulers expose fine-grained control [9, 26]. PROFGUARD instead preserves dedicated-equivalent profiling under sharing.

Kernel optimization. Autotuners rely on measured performance as the search signal, combining runtime measurement, cost models, and schedule exploration [5, 35, 28, 29]; recent LLM-based systems add execution feedback, profiling, RL, or evolutionary search for CUDA, Triton, CuTe, and accelerator kernels [21, 6, 11, 4]. They motivate our workload but assume reliable measurements; PROFGUARD preserves that reward signal while allowing overlap outside measured intervals.

7 Conclusion

We present PROFGUARD, a runtime that allows GPU tasks issued by GPU kernel coding agents to execute concurrently on a shared server while preserving the profiling accuracy of dedicated execution. PROFGUARD achieves this through *atomic profiling*: it intercepts vendor profiling interfaces to identify profiling intervals, then coordinates CPU scheduling, GPU execution, and VRAM management to protect only those intervals while allowing execution outside them to overlap across tasks.

References

[1] Anthropic. Claude code. <https://code.claude.com/docs/en/overview>, 2026.

[2] Joshua Bakita and James H. Anderson. Demystifying NVIDIA GPU internals to enable reliable GPU management. In *Proceedings of the 30th IEEE Real-Time and Embedded Technology and Applications Symposium*, 2024.

[3] ByteDance Seed. `cudaLLM-data`. <https://huggingface.co/datasets/ByteDance-Seed/cudaLLM-data>, 2025. Hugging Face dataset.

[4] Terry Chen, Zhifan Ye, Bing Xu, Zihao Ye, Timmy Liu, Ali Hassani, Tianqi Chen, Andrew Kerr, Haicheng Wu, Yang Xu, Yu-Jung Chen, Hanfeng Chen, Aditya Kane, Ronny Krashinsky, Ming-Yu Liu, Vinod Grover, Luis Ceze, Roger Bringmann, John Tran, Wei Liu, Fung Xie, Michael Lightstone, and Humphrey Shi. AVO: Agentic variation operators for autonomous evolutionary search. *arXiv preprint arXiv:2603.24517*, 2026.

[5] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. TVM: An automated End-to-End optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 578–594, Carlsbad, CA, October 2018. USENIX Association.

[6] Weinan Dai, Hanlin Wu, Qiying Yu, Huan ang Gao, Jiahao Li, Chengquan Jiang, Weiqiang Lou, Yufan Song, Hongli Yu, Jiase Chen, Wei-Ying Ma, Ya-Qin Zhang, Jingjing Liu, Mingxuan Wang, Xin Liu, and Hao Zhou. CUDA agent: Large-scale agentic RL for high-performance CUDA kernel generation. *arXiv preprint arXiv:2602.24286*, 2026.

[7] DeepSeek-AI. DeepSeek-V4: Towards highly efficient million-token context intelligence. Technical Report, 2026.

[8] Juncheng Gu, Mosharaf Chowdhury, Kang G. Shin, Yibo Zhu, Myeongjae Jeon, Junjie Qian, Hongqiang Harry Liu, and Chuanxiong Guo. Tiresias: A GPU cluster manager for distributed deep learning. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, pages 485–500. USENIX Association, 2019.

[9] Mingcong Han, Hanze Zhang, Rong Chen, and Haibo Chen. Microsecond-scale preemption for concurrent GPU-accelerated DNN inference. In *16th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2022*. USENIX Association, 2022.

[10] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, pages 611–626. ACM, 2023.

- [11] Gang Liao et al. Kernelevolve: Scaling agentic kernel coding for heterogeneous AI accelerators at meta. *arXiv preprint arXiv:2512.23236*, 2026.
- [12] Kshiteej Mahajan, Arjun Balasubramanian, Arjun Singhvi, Shivaram Venkataraman, Aditya Akella, Amar Phanishayee, and Shuchi Chawla. Themis: Fair and efficient GPU cluster scheduling. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 289–304. USENIX Association, 2020.
- [13] MiniMax. MiniMax-M3. <https://www.minimax.io/blog/minimax-m3>, 2026.
- [14] MIT HAN Lab. Kernel Design Agents. <https://github.com/mit-han-lab/kernel-design-agents>, 2026.
- [15] Deepak Narayanan, Keshav Santhanam, Amar Phanishayee, and Matei Zaharia. Heterogeneity-aware cluster scheduling policies for deep learning workloads. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 481–498. USENIX Association, 2020.
- [16] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Anand Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. Efficient large-scale language model training on GPU clusters using Megatron-LM. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2021.
- [17] NVIDIA Corporation. *CUDA Multi-Process Service Documentation*, 2026.
- [18] NVIDIA Corporation. *CUDA Toolkit Documentation*, 2026.
- [19] NVIDIA Corporation. *Efficient GEMM in CUDA*, 2026.
- [20] NVIDIA Corporation. *NVIDIA Multi-Instance GPU User Guide*, 2026.
- [21] Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. Kernel-Bench: Can LLMs write efficient GPU kernels? *arXiv preprint arXiv:2502.10517*, 2025.
- [22] PyTorch Contributors. Ahead-of-time compilation with torch.compile. https://docs.pytorch.org/docs/main/user_guide/torch_compiler/torch.compile_aot_compile.html, 2026. Accessed: 2026-06-09.
- [23] PyTorch Contributors. torch.compile api reference. <https://docs.pytorch.org/docs/2.9/generated/torch.compile.html>, 2026. Accessed: 2026-06-09.
- [24] Aurick Qiao, Sang Keun Choe, Suhas Jayaram Subramanya, Willie Neiswanger, Qirong Ho, Hao Zhang, Gregory R. Ganger, and Eric P. Xing. Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*, pages 1–18. USENIX Association, 2021.
- [25] Qwen Team. Qwen3.7. <https://qwen.ai/blog?id=qwen3.7>, 2026.
- [26] Weihang Shen, Mingcong Han, Jialong Liu, Rong Chen, and Haibo Chen. Xsched: Preemptive scheduling for diverse xpus. In *19th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2025*, pages 671–692. USENIX Association, 2025.
- [27] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. HybridFlow: A flexible and efficient RLHF framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, 2025.
- [28] Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pages 10–19. ACM, 2019.
- [29] Ben van Werkhoven. Kernel tuner: A search-optimizing GPU code auto-tuner. *Future Generation Computer Systems*, 90:347–358, 2019.
- [30] Jiali Wang, Yankui Wang, Mingcong Han, and Rong Chen. Colocating ML inference and training with fast GPU memory handover. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*, pages 1657–1675, Boston, MA, July 2025. USENIX Association.
- [31] Wencong Xiao, Romil Bhardwaj, Ramachandran Ramjee, Muthian Sivathanu, Nipun Kwatra, Zhenhua Han, Pratyush Patel, Xuan Peng, Hanyu Zhao, Quanlu Zhang, Fan Yang, and Lidong Zhou. Gandiva: Introspective cluster scheduling for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2018, Carlsbad, CA, USA, October 8-10, 2018*, pages 595–610. USENIX Association, 2018.
- [32] Wencong Xiao, Shiru Ren, Yong Li, Yang Zhang, Pengyang Hou, Zhi Li, Yihui Feng, Wei Lin, and Yangqing Jia. AntMan: Dynamic scaling on GPU clusters for deep learning. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 533–548. USENIX Association, 2020.
- [33] Peifeng Yu and Mosharaf Chowdhury. Salus: Fine-grained GPU sharing primitives for deep learning applications. In *Proceedings of Machine Learning and Systems*, 2020.
- [34] Hanyu Zhao, Zhenhua Han, Zhi Yang, Quanlu Zhang, Fan Yang, Lidong Zhou, Mao Yang, Francis C. M. Lau, Yuqi Wang, Yifan Xiong, and Bin Wang. HiveD: Sharing a GPU cluster for deep learning with guarantees. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 515–532. USENIX Association, 2020.
- [35] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, Joseph E. Gonzalez, and Ion Stoica. Ansor: Generating high-performance tensor programs for deep learning. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 863–879. USENIX Association, November 2020.
- [36] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems*, 2024.

[37] Ruidong Zhu, Mingcong Han, Yinmin Zhong, Wencong Xiao, Xuanzhe Liu, and Xin Jin. DistRS: Disaggregated reward service for RLVR with Batch-Level constraint. In *23rd USENIX*

Symposium on Networked Systems Design and Implementation (NSDI 26), pages 1517–1531, Renton, WA, May 2026. USENIX Association.